МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ И МАССОВЫХ КОММУНИКАЦИЙ

Ордена Трудового Красного Знамени
Федеральное государственное бюджетное общеобразовательное учреждение высшего образования

“Московский технический университет связи и информатики”
(МТУСИ)
Кафедра “Программная инженерия”

Лабораторная работа №6

по дисциплине “Введение в информационные технологии”

Выполнил: Студент группы
БПИ2503
Яричевский Даниил

Москва
2025

Цель работы

Овладеть основами создания графического интерфейса пользователя (GUI) с использованием библиотеки PyQt5 на примере разработки игры "Крестики-нолики".

Задачи

1.Используйте предоставленные фрагменты кода для создания игры "Крестики-нолики".

2.Дополните метод check_winner логикой определения победителя или ничьей. Определение победы происходит по строкам, столбцам и диагоналям. После завершения игры необходимо заблокировать игровое поле.

3.Реализуйте механизм, позволяющий начать новую игру после завершения текущей без перезапуска приложения

4. Добавьте защиту от некорректных действий пользователя.

5.Реализуйте подсчет статистики.

6.Реализуйте режим игры против компьютера с использованием простого алгоритма выбора хода. https://habr.com/ru/articles/329058/

Ход работы

1. Используем предоставленные фрагменты кода для создания игры "Крестики-нолики" Добавим проверку выигрыша игрока построчно и по диагонали

```
def check_winner(self):
    for i in range(3):
        if self.buttons[i][0].text() == self.buttons[i][1].text() == self.buttons[i][2].text() != "":
            return True
        if self.buttons[0][i].text() == self.buttons[1][i].text() == self.buttons[2][i].text() != "":
            return True
    # Проверка диагоналей
    if self.buttons[0][0].text() == self.buttons[1][1].text() == self.buttons[2][2].text() != "":
        return True
    if self.buttons[0][2].text() == self.buttons[1][1].text() == self.buttons[2][0].text() != "":
        return True
    return False
```

2. Дополним метод check_winner логикой определения победителя или ничьей. Определение победы происходит по строкам, столбцам и диагоналям. После завершения игры блокируется игровое поле.

```
def on_click(self):
    button = self.sender()
    if button.text() == "":
        button.setText(self.current_player)
        self.move_count += 1
        if self.check_winner():
            self.end_game(f"Игрок {self.current_player} победил!")
        elif self.move_count == 9:
            self.end_game("Ничья!")
        else:
            self.switch_player()
```

3. Реализуем механизм, позволяющий начать новую игру после завершения текущей без перезапуска приложения. Добавим кнопку в init, с помощью методов hide и show будем прятать и показывать кнопку при необходимости. Добавим функцию reset_metod который будет обнулять ходы, давать первый ход X и возвращать кнопки.

```
def __init__(self):
    super().__init__()
    self.setWindowTitle("Крестики-нолики на PyQt5")
    self.setGeometry(100, 100, 300, 320)  # X, Y, ширина, высота
    self.buttons = []
    self.current_player = "X"
    self.move_count = 0  # Счетчик ходов
    self.create_buttons()
    self.status_label = QLabel(self)
    self.status_label.setGeometry(0, 300, 300, 20)
    # Расположение под игровым полем
    self.status_label.setAlignment(Qt.AlignCenter)

    self.reset_button = QPushButton("Играть снова", self)
    self.reset_button.setGeometry(0, 0, 300, 300)
    self.reset_button.clicked.connect(self.reset_game)
    self.reset_button.hide()
```

```
def end_game(self, message):
    self.status_label.setText(message)
    for row in self.buttons:
        for button in row:
            button.setEnabled(False)
    self.reset_button.show()

def switch_player(self):
    self.current_player = "O" if self.current_player == "X" else "X"

def reset_game(self):
    self.current_player = "X"
    self.move_count = 0
    for row in self.buttons:
        for button in row:
            button.setEnabled(True)
            button.setText("")
    self.reset_button.hide()
```

4. Защита от некорректных действий пользователя уже встроена в проверке функции onclick.

```
def on_click(self):
    button = self.sender()
    if button.text() == "":
        button.setText(self.current_player)
        self.move_count += 1
        if self.check_winner():
            self.end_game(f"Игрок {self.current_player} победил!")
        elif self.move_count == 9:
            self.end_game("Ничья!")
        else:
            self.switch_player()
```

5. Реализуем подсчет статистики.

```
self.wins_x = 0
self.wins_o = 0
self.draws = 0

self.score_label = QLabel(self)
self.score_label.setGeometry(0, 330, 300, 20)
self.score_label.setAlignment(Qt.AlignCenter)
self.update_score_text()
```

```
def update_score_text(self):
    self.score_label.setText(f"X: {self.wins_x}  |  Ничьи: {self.draws}  |  O: {self.wins_o}")
```

```
def on_click(self):
    button = self.sender()
    if button.text() == "":
        button.setText(self.current_player)
        self.move_count += 1
        if self.check_winner():
            if self.current_player == "X":
                self.wins_x += 1
            else:
                self.wins_o += 1
            self.update_score_text()
            self.end_game(f"Игрок {self.current_player} победил!")
        elif self.move_count == 9:
            self.draws += 1
            self.update_score_text() # Обновляем табло
            self.end_game("Ничья!")
        else:
            self.switch_player()
```

6. Реализуйте режим игры против компьютера с использованием простого алгоритма выбора хода. https://habr.com/ru/articles/329058/

```
def computer_move(self):
    best_score = -float('inf')
    best_move = None
    for row in range(3):
        for col in range(3):
            if self.buttons[row][col].text() == "":
                self.buttons[row][col].setText("O")
                score = self.minimax(0, False)
                self.buttons[row][col].setText("")
                if score > best_score:
                    best_score = score
                    best_move = (row, col)
    if best_move:
        row, col = best_move
        self.buttons[row][col].setText("O")
        self.move_count += 1

        if self.check_winner():
            self.wins_o += 1
            self.update_score_text()
            self.end_game("Робот победил!")
            return
        elif self.move_count == 9:
            self.draws += 1
            self.update_score_text()
            self.end_game("Ничья!")
            return
        self.switch_player()
```

```
def minimax(self, depth, is_maximizing):
    """ Рекурсивная функция Минимакса """
    if self.check_winner():
        return 10 - depth if is_maximizing else depth - 10
    if self.is_board_full():
        return 0

    if is_maximizing:
        best_score = -float('inf')
        for row in range(3):
            for col in range(3):
                if self.buttons[row][col].text() == "":
                    self.buttons[row][col].setText("O")
                    score = self.minimax(depth + 1, False)
                    self.buttons[row][col].setText("")
                    best_score = max(score, best_score)
        return best_score
    else:
        best_score = float('inf')
        for row in range(3):
            for col in range(3):
                if self.buttons[row][col].text() == "":
                    self.buttons[row][col].setText("X")
                    score = self.minimax(depth + 1, True)
                    self.buttons[row][col].setText("")
                    best_score = min(score, best_score)
        return best_score
def is_board_full(self):

    for row in self.buttons:
        for button in row:
            if button.text() == "":
                return False
    return True
```

Вывод

Я овладел основами создания графического интерфейса пользователя (GUI) с использованием библиотеки PyQt5 на примере разработки игры "Крестики-нолики".